Frontend with JavaScript

Making Requests from HTML Pages to PHP APIs

Learning Goals

By the end of this lesson, you will be able to:

1. Explain why API requests enter through one Front Controller.
2. Trace a frontend request and backend response through five steps.
3. Send a GET request from JavaScript using `fetch()`.
4. Parse JSON, update the page, and display request errors.

IMPORTANT

Why Use a Front Controller?

- Every API request enters through `index.php`.
- One router validates both the URL path and HTTP method.
- Unknown paths consistently return `404`; unsupported methods return `405`.
- Shared response and error handling stays in

Frontend makes Requests to Backend

```
[HTML + JavaScript] ⟷ HTTP Requests ⟷ [PHP Server]
     (Frontend)                          (Backend)
```

- User requests a page in a **browser**
  - We can use `curl` to make the same request.

Web Server

- A **web server** (such as Nginx or Apache) receives an HTTP `request` and passes it to PHP.
    - PHP is a server-side programming language and runtime; it is not itself a web server.
    - In our examples, `php -S` starts PHP's built-in development web server.

- PHP processes the request and creates a `response`, such as **HTML** or **JSON**. The web server sends that response back to the browser.
- **Browser** renders the page
- After load, user interacts with **front end (JavaScript)**

Frontend & Backend Interaction

- The **frontend (JavaScript)** can send extra requests
  (AJAX, Fetch API) for APIs or dynamic content.
- The same **PHP backend** or other API endpoints handle these requests.

Web Application Frontend Tools We'll Use:

- HTML for structure
- CSS for styling
- JavaScript for API communication
    - Fetch API for HTTP requests

What We're Building for Web Applications

Frontend (Client / Browser)

1. **HTML Page**: User interface with buttons and forms
2. **JavaScript**: Makes HTTP requests to your PHP API

Backend (Server)

3. **PHP API**: Processes requests and responses (returns JSON).

Communication between Frontend and Backend

Frontend (`api_get_test.html`) ⟷ Backend (`index.php`)

IMPORTANT

Five-Step Request and Response Flow

1. The browser loads the HTML page.
2. JavaScript sends an API request with `fetch()`.
3. PHP receives and processes the request.
4. PHP returns a JSON response.
5. JavaScript reads the JSON and updates the page.

Before the Steps: Start the Application

1. Start the PHP server

```
php -S localhost:8000
```

Step 1: Browser Loads the Frontend

Open a web browser and access the server.

http://localhost:8000/api_get_test.html

- The api_get_test.html is downloaded to the local (client side) web browser.
- The web browser parses the HTML file and runs the JavaScript code.

IMPORTANT

Step 2: JavaScript Sends an API Request

```
fetch('/index.php/api')
```

- The JavaScript code makes requests (GET /index.php/api) to the server where it is downloaded from (http://localhost:8000).

Step 2 Comparison: Send the Same Request without JavaScript

We can directly make a GET request to the endpoint (`index.php/api`) with a query string (`a=b&c=d`) without using HTML/JavaScript.

http://localhost:8000/index.php/api?a=b&c=d

We can use a web browser, or we can use cURL to make the request.

```
curl "http://localhost:8000/index.php/api?a=b&c=d"
```

Reference: Front Controller URLs

All API URLs in this example must begin with `/index.php/`.

```
GET /index.php/api  → accepted
GET /api            → 404 Not Found
GET /other.php/api  → 404 Not Found
```

`index.php` is the single entry point. The router rejects requests that try to bypass it.

Step 3: PHP Receives the Request

Server/Backend (PHP) extracts the API information (api) from the GET request.

Step 3.1: Read the Request URI

```
$requestPath = parse_url(
    $_SERVER['REQUEST_URI'],
    PHP_URL_PATH
); // '/index.php/api'
```

`parse_url()` removes the query string (`?a=b&c=d`) before routing.

Step 3.2: Validate the Front Controller Path

- The path must begin with the Front Controller prefix `/index.php/`.

```php
$requestPath = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$frontController = '/index.php/';

if (!str_starts_with($requestPath, $frontController)) {
    sendError('Endpoint not found', 404);
}
```

Step 3 Detail: How `parse_url()` Works

- `parse_url()` breaks a URL string into its components: scheme, host, port, path, query, and fragment.
- Passing a component constant like `PHP_URL_PATH` (or `PHP_URL_QUERY`, `PHP_URL_HOST`, etc.) returns just that one part as a string, instead of the full array.
- Example: `parse_url('http://example.com/a/b?x=1', PHP_URL_QUERY)` returns `'x=1'`.

Step 3.3: Extract the Internal Route

- The external endpoint is `GET /index.php/api`.
- After validating the prefix, the router extracts the internal route name `api`.

```
$path = substr(
    $requestPath,
    strlen($frontController)
); // 'api'
```

Step 3 Result: Extract the Endpoint

- Public endpoint: `GET /index.php/api`
- Front Controller: `index.php`
- Internal route used by the `switch`: `api`

The internal route is an implementation detail, not a second public endpoint.

Step 4: PHP Returns a JSON Response

Step 4.1: Build the Base JSON Response

- In this example, the PHP server returns JSON as a response.
- We need the header, code, and JSON body.

```php
function sendJson(array $payload, int $code = 200): void {
    http_response_code($code);
    header('Content-Type: application/json');
    echo json_encode($payload, JSON_PRETTY_PRINT |
                               JSON_UNESCAPED_SLASHES);
    exit;
}
```

- By default, json_encode() in PHP will escape forward slashes (`/`) into `\/`.
- It's valid JSON either way, but can look strange in URLs.
- The JSON_UNESCAPED_SLASHES prevents it.

Step 4.2: Build a Success Response

The `sendResponse()` function uses `sendJson()` to make a response.

```
function sendResponse($data,
  string $message = 'Success',
  int $code = 200): void {
    $count = is_countable($data) ? count($data) : 1;
    sendJson([
        'success' => true,
        'message' => $message,
        'data'    => $data,
        'count'   => $count,
    ], $code);
}
```

Step 4.3: Build an Error Response

- Following the web protocol, we send a 400 error when an error occurs.

```
function sendError(string $message,
                   int $code = 400): void {
    sendJson([
        'success' => false,
        'message' => $message,
        'data'    => null,
    ], $code);
}
```

Step 4.4: Route to the Correct Response

- The Front Controller routes both the path and method. This lesson follows the `api` route.

```
switch ($path) {
    case 'api':
        if ($method !== 'GET') {
            sendError('Method not allowed for this endpoint', 405);
        }

        // API information endpoint
        $info = [
            'name' =>
              'Simple Student Management API',
            'version' => '1.0',
            'description' =>
              'A minimal API for learning PHP basics with student data',
            'endpoints' => [
                'GET /index.php/api' => 'Show this API information',
                'POST /index.php/submit' => 'Process submitted form data',
            ]
        ];
        sendResponse($info, 'Welcome to Simple Student Management API');
        break;

    case 'submit':
        if ($method !== 'POST') {
            sendError('Method not allowed for this endpoint', 405);
        }

        // Read the submitted fields and return JSON.
        // The next lesson explains this route in detail.
        break;
```

Step 4 Result: JSON Arrives at the Browser

- This is a response from the server.


```
{
    "success": true,
    "message": "Welcome to Simple Student Management API",
    "data": {
        "name": "Simple Student Management API",
        "version": "1.0",
        "description":
          "A minimal API for learning PHP basics with student data",
        "endpoints": {
            "GET /index.php/api": "Show this API information",
            "POST /index.php/submit": "Process submitted form data"
        }
    },
    "count": 4
}
```

Step 5: JavaScript Receives and Displays the Response

Step 5.1: Prepare the Page

- We need to get the response using HTML/JavaScript.
- In this example, when a button is clicked, the `getApiInfo()` JavaScript function is called.

```
<!DOCTYPE html>
<head> ... </head>
<body>
    <div class="container">
        <button onclick="getApiInfo()">Get API Information</button>
    </div>
    <script>
        const API_BASE = '/index.php';
        // Function to make API calls
        async function apiCall(endpoint) {
            ...
        }
        function getApiInfo() {
          // no `await`: we don't use apiCall()'s result here,
          // apiCall() already handles the response and updates the DOM itself.
          apiCall('/api');
        }
    </script>
</body>
</html>
```

Step 5.2: Parse the JSON Response

- We use the fetch JavaScript API to make a request.

```
// Simple GET request
fetch('/index.php/api')
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error('Error:', error));
```

Step 5.3: Distinguish `response` from `data`

- `response` has the HTTP-level object (status, headers, body stream).

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 234

{"message":"...",
 "data":{
    "name": ...,
    "endpoints":
      {"GET /index.php/api":"...",
       "POST /index.php/submit":"..."}
    }
 }
```

- data = actual parsed JSON object from the body.

```
{"message":"...",
 "data":
   {"name": ...,
   "endpoints":
     {"GET /index.php/api":"...",
      "POST /index.php/submit":"..."}
   }
}
```

Step 5.4: Use `async/await`

- We can use async/await to get the same results (preferred)

```
async function getApiInfo() {
    try {
        const response = await fetch('/index.php/api');
        const data = await response.json();
        console.log(data);
    } catch (error) {
        console.error('Error:', error);
    }
}
```

Why async/await? Cleaner, easier to read, better error handling

Quick Reference: async / await

- `async function f() { ... }`: marks `f` as asynchronous; it always returns a Promise, and lets you use `await` inside it.
- `await promise`: pauses **this function only** until `promise` settles, then gives you the resolved value (or throws on rejection).

- Calling an async function **without** `await` (like `apiCall('/api')`) does **not** block the caller: the caller moves on right away, while the async function keeps running in the background and finishes later.
- Use `await` when you need the result or must run steps in order (e.g., `fetch` → `.json()`); skip it when the async function handles everything itself.

Step 5.5: Prepare the Result Area

```
<div id="result" class="result">
  Click any button above to test the API...
</div>
```

We can use JavaScript to update the information in the block using id.

```
document.getElementById('result').textContent =
   JSON.stringify(data, null, 2);
```

IMPORTANT

Step 5.6: Request and Display JSON

JavaScript reads the JSON with `response.json()` and updates the HTML.

1. Using the `fetch` JavaScript function, it makes the request and receives the response from the server.

```
// Base URL for our PHP API
const API_BASE = '/index.php';

// Function to make API calls
async function apiCall(endpoint) {
    try {
        // Make the request
        const response = await fetch(API_BASE + endpoint);

        // Convert HTTP failures into JavaScript errors
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
```

2. Then, get the JSON from the response.

```
        const data = await response.json();

        // Display the result
        document.getElementById('result').textContent =
            JSON.stringify(data, null, 2);

    } catch (error) {
        console.error('Error:', error);
        document.getElementById('result').textContent =
            'Error: ' + error.message;
    }
}
```

Step 5.7: Trigger the API Call

- We make the API GET request using this `getApiInfo()` JavaScript function.

```
function getApiInfo() {
   apiCall('/api');  // Calls our API at /index.php/api
}
```

Step 5.8: Connect the Button to JavaScript

- HTML Button

```
<button onclick="getApiInfo()">Get API Information</button>
```

- JavaScript Handler

```
function getApiInfo() {
    apiCall('/api');  // Calls our API at /index.php/api
}
```

Complete Request Flow

1. User clicks the button
2. `getApiInfo()` function runs
3. `apiCall('/api')` makes an HTTP request to `/index.php/api`
4. PHP processes the request and returns JSON
5. JavaScript receives the response and updates the page

Making Requests from JavaScript

- We can make GET/POST/PUT requests using JavaScript.
    - We should make corresponding API handlers on the PHP side.

GET Request

- The following examples illustrate common Fetch API patterns.

  They are not implemented by the current `index.php` example, which handles only `GET /index.php/api` and `POST /index.php/submit`.

  REST-style endpoints will be implemented later.

```
const response = await fetch('/api/users');
```

POST Request (JSON)

```
const response = await fetch('/api/users', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'John Doe',
        email: 'john@example.com'
    })
});
```

PUT Request (JSON)

```
const response = await fetch('/api/users/123', {
    method: 'PUT',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'John Smith'
    })
});
```

Error Handling

- Step 5.6 already checks `response.ok` before parsing the JSON.
- `fetch()` rejects network failures, but an HTTP error such as 404 still produces a response.

```
if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
}
```

Handle Different Error Types

```
try {
    // Make the request and process the response
} catch (error) {
    const message = error.name === 'TypeError'
        ? 'Network error. Please check your connection.'
        : `Request failed: ${error.message}`;

    document.getElementById('result').textContent = message;
}
```

In the next lesson:

- JavaScript collects form fields with `FormData` and sends them as `multipart/form-data`.
- **Handle User Inputs and Forms** walks through `form_post_test.html` and the `POST /index.php/submit` route step by step.

Key Takeaways

1. **HTML** provides the user interface structure
2. **JavaScript** handles API communication and DOM updates
3. **Fetch API** makes modern HTTP requests easy
4. **async/await** provides clean asynchronous code
5. **Error handling** is crucial for a good user experience